

Optimizing Azure Cosmos DB Costs for Billing Records

Solution 1: Tiered Storage with Function Proxy (Simple Cold Storage Offload)

Executive Summary:

This solution moves all billing records older than 3 months from Azure Cosmos DB to Azure Blob Storage (Cold tier) using scheduled Azure Functions. A Function Proxy intercepts all read requests: it first checks Cosmos DB, and if the record is not found, it fetches from Blob Storage. This ensures full backward compatibility and minimal architecture change.

Key Components:

- Azure Cosmos DB: Stores hot billing records (< 3 months)
- Azure Blob Storage (Cold Tier): Stores archived records
- Azure Function (Timer): Moves cold records from Cosmos DB to Blob
- Azure Function (HTTP): Serves archived records
- Function Proxy Layer: Unified API that routes reads between Cosmos DB and Blob
- Cosmos TTL (Optional): Can be used for auto-deletion of archived records

Implementation:

1. Timer-triggered Azure Function queries Cosmos DB for records older than 3 months, writes them to Blob Storage and optionally deletes from Cosmos DB.
2. HTTP-triggered Azure Function reads from Cosmos DB first. If not found, reads from Blob.
3. Unified API ensures no changes to clients.

Pros:

- Simple to implement and manage.
- No changes to existing APIs or clients.
- Full control over migration timing.

Cons:

- Routing logic managed inside the function.
- Archival is a separate batch job.

Solution 2: Automated Hybrid Tiering with TTL, Change Feed, and API Management Routing

Executive Summary:

This is a fully automated, cloud-native data lifecycle solution using Cosmos DB Change Feed, TTL, Autoscale, and Azure API Management. Cold records are moved to Blob Storage automatically and routed intelligently via API Management.

Key Components:

- Cosmos DB (Hot Data): Stores recent (< 3-month) records
- TTL on Cosmos DB: Automatically deletes records after 90 days
- Change Feed: Streams inserts/updates/deletes to a Function
- Azure Function (Change Feed Trigger): Archives and deletes records from TTL expirations
- Azure Blob Storage (Cold Tier): Stores archived records
- API Management: Routes requests to Cosmos or cold Function

Optimizing Azure Cosmos DB Costs for Billing Records

- Azure Function (Cold Reader): Fetches archived records
- Autoscale Throughput: Dynamically adjusts RU/s

Implementation:

1. TTL in Cosmos DB automatically deletes records older than 90 days.
2. Change Feed captures inserts/updates/deletions.
3. Function writes to Blob on each change. Cold data is retrieved via HTTP-triggered function.
4. API Management routes requests to hot or cold tier based on age.
5. Enable Cosmos DB Autoscale to optimize RU/s usage.

Pros:

- Fully automated archival and cleanup.
- Real-time consistency using Change Feed.
- Transparent to API clients.
- Uses built-in TTL and autoscale for cost efficiency.

Cons:

- Slightly more complex setup.
- Cold reads require accurate routing (e.g., date inference).

Final Recommendation:

- Use Solution 1 for quick implementation and simplicity.
- Use Solution 2 for long-term cost efficiency and automation.